

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 5
процедуры, функции, триггеры в PostgreSQL

Обучающийся Сафонова Арина Дмитриевна
Факультет прикладной информатики
Группа K3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2023
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание (min - 6 баллов, max - 10 баллов, доп. баллы - 3):

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:
Вариант 2.1. 7 триггеров по ИЗ.

Практическое задание

В рамках лабораторной работы необходимо выполнить создание и проверку процедур и триггеров в индивидуальной базе данных гостиницы.

Необходимо создать три хранимые процедуры:

1. Процедуру для увеличения цены всех номеров на 5%, если в отеле нет свободных номеров.
2. Процедуру для получения информации о свободных одноместных номерах отеля на завтрашний день.
3. Процедуру для бронирования двухместного номера в гостинице на заданную дату и количество дней проживания.

Также необходимо создать семь триггеров для индивидуальной базы данных:

1. При добавлении бронирования автоматически менять статус номера на booked.
2. При отмене бронирования автоматически менять статус номера на free.
3. При завершении проживания автоматически менять статус номера на free.
4. При добавлении оплаты проверять, что сумма оплаты не превышает сумму бронирования.
5. При добавлении бронирования запрещать бронирование номера, который уже имеет статус booked или occupied.
6. При добавлении или изменении цены запрещать пересечение периодов цен для одного типа номера.
7. При добавлении или изменении акции проверять корректность процента скидки и периода действия акции.

Работа выполняется в PostgreSQL с использованием консоли psql.



Type "help" for help.

hotel_db=# \dt hotel.*

List of tables			
Schema	Name	Type	Owner
hotel	amenity	table	postgres
hotel	booking_order	table	postgres
hotel	city	table	postgres
hotel	cleaning_room	table	postgres
hotel	contract	table	postgres
hotel	employee	table	postgres
hotel	guest	table	postgres
hotel	history_position	table	postgres
hotel	hotel	table	postgres
hotel	payment	table	postgres
hotel	position	table	postgres
hotel	price	table	postgres
hotel	promotion	table	postgres
hotel	room	table	postgres
hotel	type_room	table	postgres

(15 rows)

hotel_db=#

hotel_db=# \d hotel.room

Table "hotel.room"				
Column	Type	Collation	Nullable	Default
id_room	integer		not null	
id_hotel	integer		not null	
id_type_room	integer		not null	
floor	integer		not null	
number_room	integer		not null	
status_room	character varying		not null	

Indexes:

"room_pkey" PRIMARY KEY, btree (id_room)

"fki_fk_room_hotel" btree (id_hotel)

"fki_fk_room_type_room" btree (id_type_room)

"idx_room_hotel_status" btree (id_hotel, status_room)

"uq_room_hotel_number" UNIQUE CONSTRAINT, btree (id_hotel, number_room)

Check constraints:

"chk_room_floor" CHECK (floor >= 1)

"chk_room_status" CHECK (status_room::text = ANY (ARRAY['free'::character varying, 'occupied'::character varying, 'booked'::character varying]::text[]))

Foreign-key constraints:

"fk_room_hotel" FOREIGN KEY (id_hotel) REFERENCES hotel.hotel(id_hotel)

"fk_room_type_room" FOREIGN KEY (id_type_room) REFERENCES

hotel.type_room(id_type_room)

Referenced by:

TABLE "hotel.cleaning_room" CONSTRAINT "fk_cleaning_room_room" FOREIGN KEY (id_room) REFERENCES hotel.room(id_room)

hotel_db=# SELECT

hotel_db=# h.id_hotel,

hotel_db=# h.name_hotel,

hotel_db=# r.status_room,

hotel_db=# COUNT(*) AS count_rooms

hotel_db=# FROM hotel.hotel h

```

hotel_db=# JOIN hotel.room r
hotel_db=# ON h.id_hotel = r.id_hotel
hotel_db=# GROUP BY h.id_hotel, h.name_hotel, r.status_room
hotel_db=# ORDER BY h.id_hotel, r.status_room;
id_hotel | name_hotel | status_room | count_rooms
-----+-----+-----+-----

```

```

1 | Grand Hotel | booked | 2
1 | Grand Hotel | occupied | 1
2 | Nevsky Palace | booked | 1
2 | Nevsky Palace | free | 1

```

(4 rows)

- для увеличения цены всех номеров на 5 %, если в отеле нет свободных номеров;

Скрин цен до процедуры

```

hotel_db=#
hotel_db=# SELECT
hotel_db=#     p.id_price,
hotel_db=#     p.id_room_type,
hotel_db=#     tr.name_type_room,
hotel_db=#     p.price_per_day
hotel_db=# FROM hotel.price p
hotel_db=# JOIN hotel.type_room tr
hotel_db=# ON p.id_room_type = tr.id_type_room
hotel_db=# ORDER BY p.id_price;
id_price | id_room_type | name_type_room | price_per_day
-----+-----+-----+-----
[
1 | 1 | Single | 3150.00
2 | 2 | Double | 5250.00
3 | 3 | Family | 8400.00
(3 rows)

```

Процедура

```

CREATE
OR REPLACE PROCEDURE hotel.increase_price_if_no_free_rooms(p_hotel_id integer) LANGUAGE
plpgsql AS $$ BEGIN IF NOT EXISTS (
SELECT
1
FROM hotel.room
WHERE id_hotel = p_hotel_id
AND status_room = 'free'
) THEN
UPDATE hotel.price
SET
price_per_day = price_per_day * 1.05
WHERE id_room_type IN (
SELECT
DISTINCT id_type_room
FROM hotel.room
WHERE id_hotel = p_hotel_id
);
RAISE NOTICE 'В отеле нет свободных номеров. Цены увеличены на 5 процентов.';

```

```

ELSE RAISE NOTICE 'В отеле есть свободные номера. Цены не изменены.';
END IF;
END;
$$;

```

```

arinasafonova — psql — runpsql.sh — 121x28
hotel_db=# CREATE OR REPLACE PROCEDURE hotel.increase_price_if_no_free_rooms(
hotel_db(#   p_hotel_id integer
hotel_db(# )
hotel_db=# LANGUAGE plpgsql
hotel_db=# AS $$
hotel_db$# BEGIN
hotel_db$#     IF NOT EXISTS (
hotel_db$#         SELECT 1
hotel_db$#         FROM hotel.room
hotel_db$#         WHERE id_hotel = p_hotel_id
hotel_db$#         AND status_room = 'free'
hotel_db$#     ) THEN
hotel_db$#         UPDATE hotel.price
hotel_db$#         SET price_per_day = price_per_day * 1.05
hotel_db$#         WHERE id_room_type IN (
hotel_db$#             SELECT DISTINCT id_type_room
hotel_db$#             FROM hotel.room
hotel_db$#             WHERE id_hotel = p_hotel_id
hotel_db$#         );
hotel_db$#         RAISE NOTICE 'В отеле нет свободных номеров. Цены увеличены на 5 процентов.';
hotel_db$#     ELSE
hotel_db$#         RAISE NOTICE 'В отеле есть свободные номера. Цены не изменены.';
hotel_db$#     END IF;
hotel_db$# END;
hotel_db$# $$;
CREATE PROCEDURE
hotel_db=#

```

Вызов процедуры и состояние после

NOTICE: В отеле нет свободных номеров. Цены увеличены на 5 процентов.

```

CREATE PROCEDURE
hotel_db=# \df hotel.increase_price_if_no_free_rooms
                                List of functions
Schema | Name | Result data type | Argument data types | Type
-----+-----+-----+-----+-----
hotel | increase_price_if_no_free_rooms | | IN p_hotel_id integer | proc
(1 row)

hotel_db=# CALL hotel.increase_price_if_no_free_rooms(1);
NOTICE: В отеле нет свободных номеров. Цены увеличены на 5 процентов.
CALL
hotel_db=# SELECT
hotel_db=#     p.id_price,
hotel_db=#     p.id_room_type,
hotel_db=#     tr.name_type_room,
hotel_db=#     p.price_per_day
hotel_db=# FROM hotel.price p
hotel_db=# JOIN hotel.type_room tr
hotel_db=#     ON p.id_room_type = tr.id_type_room
hotel_db=# ORDER BY p.id_price;
 id_price | id_room_type | name_type_room | price_per_day
-----+-----+-----+-----
1 | 1 | Single | 3307.50
2 | 2 | Double | 5512.50
3 | 3 | Family | 8820.00
(3 rows)

hotel_db=#

```

Процедура увеличения цены номеров при отсутствии свободных номеров

Была создана хранимая процедура `hotel.increase_price_if_no_free_rooms`, предназначенная для увеличения стоимости проживания на 5% для всех типов номеров заданного отеля в случае, если в этом отеле отсутствуют свободные номера.

Процедура принимает входной параметр `p_hotel_id`, в который передаётся идентификатор отеля. Внутри процедуры выполняется проверка наличия свободных номеров в таблице `hotel.room`. Если для заданного отеля не найдено ни одного номера со статусом `free`, то выполняется обновление таблицы `hotel.price`: значение

поля price_per_day увеличивается на 5% для типов номеров, представленных в данном отеле.

Если свободные номера в отеле есть, изменение цен не выполняется, а пользователю выводится соответствующее сообщение.

После создания процедура была вызвана командой:

```
CALL hotel.increase_price_if_no_free_rooms(1);
```

В результате выполнения процедуры было получено сообщение:

NOTICE: В отеле нет свободных номеров. Цены увеличены на 5 процентов.

После вызова процедуры была выполнена проверка таблицы hotel.price. Значения цен увеличились на 5%, что подтверждает корректную работу процедуры.

- для получения информации о свободных одноместных номерах отеля на завтрашний день;

Данные ДО процедуры №2

```
SELECT
  h.name_hotel,
  r.number_room,
  r.floor,
  tr.name_type_room,
  tr.count_places,
  r.status_room
FROM hotel.room r
JOIN hotel.hotel h
ON r.id_hotel = h.id_hotel
JOIN hotel.type_room tr
ON r.id_type_room = tr.id_type_room
WHERE h.id_hotel = 2
  AND tr.count_places = 1
  AND r.status_room = 'free'
  AND NOT EXISTS (
    SELECT
      1
    FROM hotel.booking_order b
    WHERE b.id_room = r.id_room
      AND b.status IN ('created', 'confirmed')
      AND CURRENT_DATE + 1 >= b.check_in_date
      AND CURRENT_DATE + 1 < b.check_out_date
  );
```

```

hotel_db=# SELECT
hotel_db=#     h.name_hotel,
hotel_db=#     r.number_room,
hotel_db=#     r.floor,
hotel_db=#     tr.name_type_room,
hotel_db=#     tr.count_places,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=#     ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=#     ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE h.id_hotel = 2
hotel_db=#     AND tr.count_places = 1
hotel_db=#     AND r.status_room = 'free'
hotel_db=#     AND NOT EXISTS (
hotel_db(#         SELECT 1
hotel_db(#         FROM hotel.booking_order b
hotel_db(#         WHERE b.id_room = r.id_room
hotel_db(#             AND b.status IN ('created', 'confirmed')
hotel_db(#             AND CURRENT_DATE + 1 >= b.check_in_date
hotel_db(#             AND CURRENT_DATE + 1 < b.check_out_date
hotel_db(#     );
name_hotel | number_room | floor | name_type_room | count_places | status_room
-----+-----+-----+-----+-----+-----
Nevsky Palace | 101 | 1 | Single | 1 | free
(1 row)

hotel_db=#

```

Процедура №2

```

CREATE
OR REPLACE PROCEDURE hotel.get_free_single_rooms_tomorrow(p_hotel_id integer) LANGUAGE
plpgsql AS $$ DECLARE rec record;
BEGIN RAISE NOTICE 'Свободные одноместные номера отеля на завтрашний день:';
FOR rec IN
SELECT
    h.name_hotel,
    r.number_room,
    r.floor,
    tr.name_type_room,
    tr.count_places,
    r.status_room
FROM hotel.room r
JOIN hotel.hotel h
ON r.id_hotel = h.id_hotel
JOIN hotel.type_room tr
ON r.id_type_room = tr.id_type_room
WHERE h.id_hotel = p_hotel_id
    AND tr.count_places = 1
    AND r.status_room = 'free'
    AND NOT EXISTS (
        SELECT
            1
        FROM hotel.booking_order b
        WHERE b.id_room = r.id_room
            AND b.status IN ('created', 'confirmed')
            AND CURRENT_DATE + 1 >= b.check_in_date
            AND CURRENT_DATE + 1 < b.check_out_date
    ) LOOP RAISE NOTICE 'Отель: %, номер: %, этаж: %, тип: %, мест: %, статус: %',
        rec.name_hotel,
        rec.number_room,
        rec.floor,
        rec.name_type_room,
        rec.count_places,
        rec.status_room;
END LOOP;
END;
$$;

```

Процедура получения свободных одноместных номеров на завтрашний день

Была создана процедура `hotel.get_free_single_rooms_tomorrow`, которая выводит информацию о свободных одноместных номерах заданного отеля на завтрашний день. Перед созданием процедуры был выполнен запрос для проверки исходных данных. Был найден свободный одноместный номер 101 в отеле Nevsky Palace. Процедура принимает параметр `p_hotel_id`, выбирает номера с `count_places = 1` и статусом `free`, а также проверяет отсутствие активного бронирования на завтрашний день. После вызова процедуры:
`CALL hotel.get_free_single_rooms_tomorrow(2);`
был получен результат:
NOTICE: Отель: Nevsky Palace, номер: 101, этаж: 1, тип: Single, мест: 1, статус: free
Таким образом, процедура корректно выводит свободные одноместные номера отеля на завтрашний день.

- бронирования двухместного номера в гостинице на заданную дату и количество дней проживания.

состояние ДО бронирования: какие двухместные номера есть и какие бронирования уже существуют

(3 rows)

```
hotel_db=# SELECT
hotel_db=#   h.id_hotel,
hotel_db=#   h.name_hotel,
hotel_db=#   r.id_room,
hotel_db=#   r.number_room,
hotel_db=#   tr.name_type_room,
hotel_db=#   tr.count_places,
hotel_db=#   r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=#   ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=#   ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE tr.count_places = 2
hotel_db=# ORDER BY h.id_hotel, r.number_room;
 id_hotel | name_hotel | id_room | number_room | name_type_room | count_places |
status_room
```

```
-----+-----+-----+-----+-----+-----+-----
      1 | Grand Hotel | 2 | 102 | Double | 2 | booked
      2 | Nevsky Palace | 5 | 202 | Double | 2 | booked
```

(2 rows)

```
hotel_db=# SELECT
hotel_db=#   id_order,
hotel_db=#   id_guest,
hotel_db=#   id_employee,
hotel_db=#   id_room,
hotel_db=#   check_in_date,
hotel_db=#   check_out_date,
hotel_db=#   status,
hotel_db=#   amount
```



```

hotel_db=# FROM hotel.booking_order
hotel_db=# ORDER BY id_order;
 id_order | id_guest | id_employee | id_room | check_in_date | check_out_date | status | amount

```

```

-----+-----+-----+-----+-----+-----+-----+-----
      1 |      1 |      1 |      1 | 2026-05-20 | 2026-05-24 | confirmed | 10000
      2 |      2 |      1 |      4 | 2026-05-19 | 2026-05-23 | created   | 8000
      3 |      3 |      3 |      | 2024-08-05 | 2024-08-09 | completed | 16000

```

(3 rows)

```

hotel_db=#

```

```

CREATE
  OR REPLACE PROCEDURE hotel.get_free_single_rooms_tomorrow(p_hotel_id integer) LANGUAGE
plpgsql AS $$ DECLARE rec record;
BEGIN RAISE NOTICE 'Свободные одноместные номера отеля на завтрашний день:';
FOR rec IN
SELECT
  h.name_hotel,
  r.number_room,
  r.floor,
  tr.name_type_room,
  tr.count_places,
  r.status_room
FROM hotel.room r
JOIN hotel.hotel h
ON r.id_hotel = h.id_hotel
JOIN hotel.type_room tr
ON r.id_type_room = tr.id_type_room
WHERE h.id_hotel = p_hotel_id
  AND tr.count_places = 1
  AND r.status_room = 'free'
  AND NOT EXISTS (
    SELECT
      1
    FROM hotel.booking_order b
    WHERE b.id_room = r.id_room
      AND b.status IN ('created', 'confirmed')
      AND CURRENT_DATE + 1 >= b.check_in_date
      AND CURRENT_DATE + 1 < b.check_out_date
  ) LOOP RAISE NOTICE 'Отель: %, номер: %, этаж: %, тип: %, мест: %, статус: %',
  rec.name_hotel,
  rec.number_room,
  rec.floor,
  rec.name_type_room,
  rec.count_places,
  rec.status_room;
END LOOP;
END;
$$;

```

Вызов процедуры и результат

```

hotel_db=# CALL hotel.get_free_single_rooms_tomorrow(2);
NOTICE: Свободные одноместные номера отеля на завтрашний день:
NOTICE: Отель: Nevsky Palace, номер: 101, этаж: 1, тип: Single, мест: 1, статус: free
CALL
hotel_db=#

```

```

hotel_db=# $$;
CREATE PROCEDURE
hotel_db=# \df hotel.get_free_single_rooms_tomorrow
List of functions
Schema | Name | Result data type | Argument data types | Type
-----+-----+-----+-----+-----
hotel | get_free_single_rooms_tomorrow | | IN p_hotel_id integer | proc
(1 row)

```

Была создана процедура `hotel.get_free_single_rooms_tomorrow`, которая выводит свободные одноместные номера заданного отеля на завтрашний день. Процедура принимает идентификатор отеля `p_hotel_id`, выбирает номера с количеством мест 1 и статусом `free`, а также проверяет отсутствие активных бронирований на завтрашнюю дату. После вызова процедуры командой `CALL hotel.get_free_single_rooms_tomorrow(2)`; был найден свободный одноместный номер 101 в отеле `Nevsky Palace`. Таким образом, процедура корректно выполняет поиск свободных одноместных номеров на завтрашний день.

- бронирования двухместного номера в гостинице на заданную дату и количество дней проживания.

```

hotel_db=# UPDATE hotel.room
hotel_db=# SET status_room = 'free'
hotel_db=# WHERE id_room = 5;
UPDATE 1
hotel_db=# SELECT
hotel_db=#     h.id_hotel,
hotel_db=#     h.name_hotel,
hotel_db=#     r.id_room,
hotel_db=#     r.number_room,
hotel_db=#     tr.name_type_room,
hotel_db=#     tr.count_places,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=#     ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=#     ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE tr.count_places = 2
hotel_db=# ORDER BY h.id_hotel, r.number_room;
 id_hotel | name_hotel | id_room | number_room | name_type_room | count_places | status_room
-----+-----+-----+-----+-----+-----+-----
        1 | Grand Hotel |        2 |         102 | Double         |             2 | booked
        2 | Nevsky Palace |        5 |         202 | Double         |             2 | free
(2 rows)
hotel_db=#

```

Процедура бронирования двухместного номера

```

CREATE
OR REPLACE PROCEDURE hotel.book_double_room (
    p_hotel_id INTEGER,
    p_guest_id INTEGER,
    p_employee_id INTEGER,
    p_check_in_date date,
    p_days INTEGER
) LANGUAGE plpgsql AS $$
DECLARE
    v_room_id integer;
    v_price_per_day numeric;
    v_new_order_id integer;
BEGIN
    IF p_days <= 0 THEN

```

```

        RAISE NOTICE 'Количество дней должно быть больше 0.';
    RETURN;
END IF;

SELECT
    r.id_room,
    p.price_per_day
INTO
    v_room_id,
    v_price_per_day
FROM hotel.room r
JOIN hotel.type_room tr
    ON r.id_type_room = tr.id_type_room
JOIN hotel.price p
    ON tr.id_type_room = p.id_room_type
WHERE r.id_hotel = p_hotel_id
    AND tr.count_places = 2
    AND r.status_room = 'free'
    AND NOT EXISTS (
        SELECT 1
        FROM hotel.booking_order b
        WHERE b.id_room = r.id_room
            AND b.status IN ('created', 'confirmed')
            AND p_check_in_date < b.check_out_date
            AND p_check_in_date + p_days > b.check_in_date
    )
ORDER BY r.number_room
LIMIT 1;

IF v_room_id IS NULL THEN
    RAISE NOTICE 'Свободных двухместных номеров на заданные даты нет.';
    RETURN;
END IF;

SELECT COALESCE(MAX(id_order), 0) + 1
INTO v_new_order_id
FROM hotel.booking_order;

INSERT INTO hotel.booking_order (
    id_order,
    payment_type,
    id_guest,
    id_employee,
    amount,
    check_in_date,
    check_out_date,
    status,
    id_room
)
VALUES (
    v_new_order_id,
    'card',
    p_guest_id,
    p_employee_id,
    v_price_per_day * p_days,
    p_check_in_date,
    p_check_in_date + p_days,
    'created',
    v_room_id
);

UPDATE hotel.room
SET status_room = 'booked'

```

```

WHERE id_room = v_room_id;

RAISE NOTICE 'Бронирование создано. Номер бронирования: %, номер комнаты id: %, сумма:
%',
    v_new_order_id,
    v_room_id,
    v_price_per_day * p_days;
END;
$$;

```

Результат

```

hotel_db=# CALL hotel.book_double_room(2, 1, 1, DATE '2026-06-15', 3);
NOTICE: Бронирование создано. Номер бронирования: 4, номер комнаты id: 5, сумма:
16537.50

```

```
CALL
```

```
hotel_db=#
```

```

CALL
hotel_db=# SELECT
hotel_db=#   id_order,
hotel_db=#   id_guest,
hotel_db=#   id_employee,
hotel_db=#   id_room,
hotel_db=#   check_in_date,
hotel_db=#   check_out_date,
hotel_db=#   status,
hotel_db=#   amount
hotel_db=# FROM hotel.booking_order
hotel_db=# ORDER BY id_order;
 id_order | id_guest | id_employee | id_room | check_in_date | check_out_date | status | amount
-----+-----+-----+-----+-----+-----+-----+-----
      1 |      1 |          1 |        1 | 2026-05-20   | 2026-05-24   | confirmed |    10000
      2 |      2 |          1 |        4 | 2026-05-19   | 2026-05-23   | created  |     8000
      3 |      3 |          3 |        3 | 2024-08-05   | 2024-08-09   | completed |    16000
      4 |      1 |          1 |        5 | 2026-06-15   | 2026-06-18   | created  |   16537.50
(4 rows)
hotel_db=#

```

```
CALL
```

```

hotel_db=# SELECT
hotel_db=#   id_order,
hotel_db=#   id_guest,
hotel_db=#   id_employee,
hotel_db=#   id_room,
hotel_db=#   check_in_date,
hotel_db=#   check_out_date,
hotel_db=#   status,
hotel_db=#   amount
hotel_db=# FROM hotel.booking_order
hotel_db=# ORDER BY id_order;
 id_order | id_guest | id_employee | id_room | check_in_date | check_out_date | status | amount
-----+-----+-----+-----+-----+-----+-----+-----

```

```

-----+-----+-----+-----+-----+-----+-----+-----
      1 |      1 |          1 |        1 | 2026-05-20   | 2026-05-24   | confirmed |    10000
      2 |      2 |          1 |        4 | 2026-05-19   | 2026-05-23   | created  |     8000
      3 |      3 |          3 |        3 | 2024-08-05   | 2024-08-09   | completed |    16000
      4 |      1 |          1 |        5 | 2026-06-15   | 2026-06-18   | created  |   16537.50
(4 rows)

```

```
hotel_db=#
```

Проверка что номер добавлен

```

hotel_db=# SELECT
hotel_db=#   h.name_hotel,

```

```

hotel_db=# r.id_room,
hotel_db=# r.number_room,
hotel_db=# tr.name_type_room,
hotel_db=# r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=# ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=# ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE r.id_room = 5;
 name_hotel | id_room | number_room | name_type_room | status_room
-----+-----+-----+-----+-----
Nevsky Palace | 5 | 202 | Double | booked
(1 row)

```

hotel_db=#

Процедура бронирования двухместного номера

Была создана процедура `hotel.book_double_room`, предназначенная для бронирования двухместного номера в заданном отеле на указанную дату и количество дней проживания. Перед проверкой процедуры номер 202 в отеле Nevsky Palace был переведён в статус `free`, чтобы его можно было использовать для тестового бронирования. После этого был выполнен запрос, подтверждающий наличие свободного двухместного номера.

Процедура принимает параметры: идентификатор отеля, гостя, сотрудника, дату заезда и количество дней проживания. Внутри процедуры выполняется поиск свободного двухместного номера, расчёт стоимости проживания, добавление новой записи в таблицу `hotel.booking_order` и изменение статуса номера на `booked`.

После вызова процедуры командой:

```
CALL hotel.book_double_room(2, 1, 1, DATE '2026-06-15', 3);
```

было создано бронирование с номером `id_order = 4`, номер комнаты `id_room = 5`, сумма проживания составила 16537.50.

Проверка таблицы `hotel.booking_order` показала появление новой записи бронирования, а проверка таблицы `hotel.room` показала, что номер 202 изменил статус на `booked`. Таким образом, процедура работает корректно.

Создание триггеров

1. При добавлении бронирования менять статус номера на `booked`
 2. При отмене бронирования менять статус номера на `free`
 3. При завершении проживания менять статус номера на `free`.
 4. При добавлении оплаты проверять, что сумма оплаты не больше суммы бронирования.
 5. При добавлении бронирования запрещать бронирование номера, который уже имеет статус `booked or occupied`
 6. При добавлении цены проверять, чтобы для одного типа номера не было пересекающихся периодов цен.
 7. При добавлении акции проверять корректность периода и процента скидки.
- При добавлении бронирования менять статус номера на `booked`.

```

hotel_db=# CREATE OR REPLACE FUNCTION hotel.fn_set_room_booked_after_booking()
hotel_db=# RETURNS trigger
hotel_db=# LANGUAGE plpgsql
hotel_db=# AS $$
hotel_db=# BEGIN
hotel_db=#     IF NEW.id_room IS NOT NULL
hotel_db=#     AND NEW.status IN ('created', 'confirmed') THEN
hotel_db=#         UPDATE hotel.room
hotel_db=#         SET status_room = 'booked'
hotel_db=#         WHERE id_room = NEW.id_room;
hotel_db=#     END IF;
hotel_db=#     RETURN NEW;
hotel_db=# END;
hotel_db=# $$;
CREATE FUNCTION
hotel_db=#

```

otel_db = # CREATE

OR REPLACE FUNCTION hotel.fn_set_room_booked_after_booking() hotel_db - # RETURNS trigger
hotel_db - # LANGUAGE plpgsql hotel_db - # AS \$\$ hotel_db\$ # BEGIN hotel_db\$ # IF NEW.id_room IS
NOT NULL hotel_db\$ #

AND NEW.status IN ('created', 'confirmed') THEN hotel_db\$ # hotel_db\$ #

UPDATE hotel.room hotel_db\$ #

SET

status_room = 'booked' hotel_db\$ #

WHERE id_room = NEW.id_room;

hotel_db\$ # hotel_db\$ # END IF;

hotel_db\$ # hotel_db\$ # RETURN NEW;

hotel_db\$ # END;

hotel_db\$ # \$\$;

CREATE FUNCTION

Создание триггера

```

A hotel_db=# CREATE TRIGGER trg_set_room_booked_after_booking
hotel_db=# AFTER INSERT ON hotel.booking_order
hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_set_room_booked_after_booking();
CREATE TRIGGER
hotel_db=#

```

hotel_db=# CREATE TRIGGER trg_set_room_booked_after_booking
hotel_db=# AFTER INSERT ON hotel.booking_order
hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_set_room_booked_after_booking();
CREATE TRIGGER
hotel_db=#

Состояние ДО проверки триггера

```

SELECT
    h.name_hotel,
    r.id_room,
    r.number_room,
    tr.name_type_room,
    r.status_room
FROM hotel.room r

```

```

JOIN hotel.hotel h
  ON r.id_hotel = h.id_hotel
JOIN hotel.type_room tr
  ON r.id_type_room = tr.id_type_room
WHERE r.id_room = 4;

```

```

hotel_db=# SELECT
hotel_db=#   h.name_hotel,
hotel_db=#   r.id_room,
hotel_db=#   r.number_room,
hotel_db=#   tr.name_type_room,
hotel_db=#   r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=#   ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=#   ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE r.id_room = 4;
  name_hotel | id_room | number_room | name_type_room | status_room
-----+-----+-----+-----+-----
Nevsky Palace |    4 |    101 | Single | free
(1 row)

```

```
hotel_db=#
```

Статус free:

```

hotel_db=# SELECT
hotel_db=#   h.name_hotel,
hotel_db=#   r.id_room,
hotel_db=#   r.number_room,
hotel_db=#   tr.name_type_room,
hotel_db=#   r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=#   ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=#   ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE r.id_room = 4;
  name_hotel | id_room | number_room | name_type_room | status_room
-----+-----+-----+-----+-----
Nevsky Palace |    4 |    101 | Single | free
(1 row)

```

Новое бронирование

```

hotel_db=# INSERT INTO hotel.booking_order (
hotel_db=#   id_order,
hotel_db=#   payment_type,
hotel_db=#   id_guest,
hotel_db=#   id_employee,
hotel_db=#   amount,
hotel_db=#   check_in_date,
hotel_db=#   check_out_date,
hotel_db=#   status,
hotel_db=#   id_room
hotel_db=# )
hotel_db=# VALUES (
hotel_db=#   5,
hotel_db=#   'card',
hotel_db=#   2,
hotel_db=#   1,
hotel_db=#   3307.50,
hotel_db=#   DATE '2026-06-20',
hotel_db=#   DATE '2026-06-21',
hotel_db=#   'created',
hotel_db=#   4
hotel_db=# );
INSERT 0 1
hotel_db=#

```

```

INSERT INTO hotel.booking_order (
  id_order,

```

```

        payment_type,
        id_guest,
        id_employee,
        amount,
        check_in_date,
        check_out_date,
        status,
        id_room
    )
VALUES (
    5,
    'card',
    2,
    1,
    3307.50,
    DATE '2026-06-20',
    DATE '2026-06-21',
    'created',
    4
);

```

status_room стал booked:

```

hotel_db=# SELECT
hotel_db=#     h.name_hotel,
hotel_db=#     r.id_room,
hotel_db=#     r.number_room,
hotel_db=#     tr.name_type_room,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=#     ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=#     ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE r.id_room = 4;
 name_hotel | id_room | number_room | name_type_room | status_room
-----
Nevsky Palace |      4 |      101 | Single | booked
(1 row)

```

Триггер 2 из 7

- при отмене бронирования номер автоматически становится *free*

Триггерная функция

```

CREATE
OR REPLACE FUNCTION hotel.fn_set_room_free_after_cancel_booking() RETURNS
trigger LANGUAGE plpgsql AS $$ BEGIN IF NEW.id_room IS NOT NULL
AND NEW.status = 'cancelled'
AND OLD.status <> 'cancelled' THEN
UPDATE hotel.room
SET
    status_room = 'free'
WHERE id_room = NEW.id_room;
END IF;
RETURN NEW;
END;
$$;

```

Создание триггера

```

CREATE TRIGGER trg_set_room_free_after_cancel_booking
AFTER UPDATE OF status ON hotel.booking_order
FOR EACH ROW

```



```
EXECUTE PROCEDURE hotel.fn_set_room_free_after_cancel_booking();
```

```
hotel_db=# CREATE TRIGGER trg_set_room_free_after_cancel_booking
hotel_db=# AFTER UPDATE OF status ON hotel.booking_order
hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_set_room_free_after_cancel_booking();
CREATE TRIGGER
hotel_db=#
```

Проверка, что триггер добавлен

```
CREATE TRIGGER
hotel_db=# \d hotel.booking_order
Table "hotel.booking_order"
  Column      | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
id_order      | integer       |           | not null |
payment_type  | character varying |           | not null |
id_guest      | integer       |           | not null |
id_employee   | integer       |           | not null |
amount        | numeric       |           |          |
check_in_date | date          |           | not null |
check_out_date| date          |           |          |
status        | character varying |           | not null |
id_room       | integer       |           |          |
Indexes:
    "booking_order_pkey" PRIMARY KEY, btree (id_order)
Check constraints:
    "chk_booking_order_amount" CHECK (amount IS NULL OR amount >= 0::numeric)
    "chk_booking_order_dates" CHECK (check_in_date < check_out_date)
    "chk_booking_order_payment_type" CHECK (payment_type::text = ANY (ARRAY['cash'::character varying, 'card'::character
    varying, 'transfer'::character varying]::text[]))
    "chk_booking_order_status" CHECK (status::text = ANY (ARRAY['created'::character varying, 'confirmed'::character var
    ying, 'cancelled'::character varying, 'completed'::character varying]::text[]))
Foreign-key constraints:
    "fk_booking_order_employee" FOREIGN KEY (id_employee) REFERENCES hotel.employee(id_employee)
    "fk_booking_order_guest" FOREIGN KEY (id_guest) REFERENCES hotel.guest(id_guest)
Referenced by:
    TABLE "hotel.amenity" CONSTRAINT "fk_amenity_order" FOREIGN KEY (id_order) REFERENCES hotel.booking_order(id_order)
    TABLE "hotel.payment" CONSTRAINT "fk_payment_order" FOREIGN KEY (id_order) REFERENCES hotel.booking_order(id_order)
Triggers:
    trg_set_room_booked_after_booking AFTER INSERT ON hotel.booking_order FOR EACH ROW EXECUTE FUNCTION hotel.fn_set_roo
    m_booked_after_booking()
    trg_set_room_free_after_cancel_booking AFTER UPDATE OF status ON hotel.booking_order FOR EACH ROW EXECUTE FUNCTION h
    otel.fn_set_room_free_after_cancel_booking()
```

Триггер рабочий

- было бронирование id_order = 5;
- ты изменила его статус на cancelled;
- после этого номер id_room = 4 автоматически стал free.

```
hotel_db=# UPDATE hotel.booking_order
hotel_db=# SET status = 'cancelled'
hotel_db=# WHERE id_order = 5;
UPDATE 1
hotel_db=# SELECT
hotel_db=#     h.name_hotel,
hotel_db=#     r.id_room,
hotel_db=#     r.number_room,
hotel_db=#     tr.name_type_room,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.room r
hotel_db=# JOIN hotel.hotel h
hotel_db=#     ON r.id_hotel = h.id_hotel
hotel_db=# JOIN hotel.type_room tr
hotel_db=#     ON r.id_type_room = tr.id_type_room
hotel_db=# WHERE r.id_room = 4;
 name_hotel | id_room | number_room | name_type_room | status_room
-----+-----+-----+-----+-----
Nevsky Palace | 4 | 101 | Single | free
(1 row)
```

Триггер 2. Освобождение номера при отмене бронирования

Был создан триггер `trg_set_room_free_after_cancel_booking`, который срабатывает при изменении статуса бронирования в таблице `hotel.booking_order`.

Если статус бронирования изменяется на `cancelled`, то триггерная функция `hotel.fn_set_room_free_after_cancel_booking()` автоматически изменяет статус соответствующего номера в таблице `hotel.room` на `free`.

Для проверки работы триггера был изменён статус бронирования `id_order = 5` на `cancelled`.

После выполнения запроса номер `id_room = 4` автоматически получил статус `free`.

Таким образом, триггер корректно освобождает номер при отмене бронирования.

Триггер 3 из 7

- при завершении проживания номер становится `free`

если статус бронирования меняется на `completed`, то номер освобождается.

Триггерная функция

CREATE

OR REPLACE FUNCTION `hotel.fn_set_room_free_after_complete_booking()` RETURNS trigger
LANGUAGE plpgsql AS \$\$ BEGIN IF NEW.id_room IS NOT NULL

AND NEW.status = 'completed'

AND OLD.status <> 'completed' THEN

UPDATE `hotel.room`

SET

`status_room = 'free'`

WHERE `id_room = NEW.id_room;`

END IF;

RETURN NEW;

END;

\$\$;

Создание триггера

```
hotel_db=# CREATE TRIGGER trg_set_room_free_after_complete_booking  
hotel_db=# AFTER UPDATE OF status ON hotel.booking_order  
hotel_db=# FOR EACH ROW  
hotel_db=# EXECUTE PROCEDURE hotel.fn_set_room_free_after_complete_booking();  
CREATE TRIGGER
```

`hotel_db=# CREATE TRIGGER trg_set_room_free_after_complete_booking`

`hotel_db=# AFTER UPDATE OF status ON hotel.booking_order`

```

hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_set_room_free_after_complete_booking());

CREATE TRIGGER

```

Проверка, что триггер в таблице

```

hotel_db=# \d hotel.booking_order
...
Triggers:
    trg_set_room_booked_after_booking AFTER INSERT ON hotel.booking_order FOR EACH
ROW EXECUTE FUNCTION hotel.fn_set_room_booked_after_booking()
    trg_set_room_free_after_cancel_booking AFTER UPDATE OF status ON
hotel.booking_order FOR EACH ROW EXECUTE FUNCTION
hotel.fn_set_room_free_after_cancel_booking()
    trg_set_room_free_after_complete_booking AFTER UPDATE OF status ON
hotel.booking_order FOR EACH ROW EXECUTE FUNCTION
hotel.fn_set_room_free_after_complete_booking()

```

Номер ДО триггера

```

hotel_db=# SELECT
hotel_db=#     b.id_order,
hotel_db=#     b.status AS booking_status,
hotel_db=#     r.id_room,
hotel_db=#     r.number_room,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.booking_order b
hotel_db=# JOIN hotel.room r
hotel_db=#     ON b.id_room = r.id_room
[hotel_db=# WHERE b.id_order = 4;
 id_order | booking_status | id_room | number_room | status_room
-----+-----+-----+-----+-----
      4 | created       |      5 |         202 | booked
(1 row)

```

```

hotel_db=# SELECT
hotel_db=#     b.id_order,
hotel_db=#     b.status AS booking_status,
hotel_db=#     r.id_room,
hotel_db=#     r.number_room,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.booking_order b
hotel_db=# JOIN hotel.room r
hotel_db=#     ON b.id_room = r.id_room
hotel_db=# WHERE b.id_order = 4;
 id_order | booking_status | id_room | number_room | status_room
-----+-----+-----+-----+-----
      4 | created       |      5 |         202 | booked
(1 row)

```

```

hotel_db=# SELECT
hotel_db=#     b.id_order,
hotel_db=#     b.status AS booking_status,
hotel_db=#     r.id_room,
hotel_db=#     r.number_room,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.booking_order b
hotel_db=# JOIN hotel.room r
hotel_db=#     ON b.id_room = r.id_room
[hotel_db=# WHERE b.id_order = 4;
 id_order | booking_status | id_room | number_room | status_room
-----+-----+-----+-----+-----
         4 | created       |        5 |         202 | booked
(1 row)

```

```

SELECT
    b.id_order,
    b.status AS booking_status,
    r.id_room,
    r.number_room,
    r.status_room
FROM hotel.booking_order b
JOIN hotel.room r
ON b.id_room = r.id_room
WHERE b.id_order = 4;
UPDATE booking_status = completed

```

```

hotel_db=# UPDATE hotel.booking_order
hotel_db=# SET status = 'completed'
[hotel_db=# WHERE id_order = 4;

```

Триггер 3. Освобождение номера при завершении проживания

Был создан триггер `trg_set_room_free_after_complete_booking`, который срабатывает при изменении статуса бронирования в таблице `hotel.booking_order`.

Если статус бронирования изменяется на `completed`, то триггерная функция `hotel.fn_set_room_free_after_complete_booking()` автоматически изменяет статус соответствующего номера в таблице `hotel.room` на `free`.

Для проверки работы триггера было выбрано бронирование `id_order = 4`. До изменения статуса бронирования номер `id_room = 5` имел статус `booked`.

После изменения статуса бронирования на `completed` номер `id_room = 5` автоматически получил статус `free`.

Таким образом, триггер корректно освобождает номер после завершения проживания.

```

hotel_db=# SELECT
hotel_db=#     b.id_order,
hotel_db=#     b.status AS booking_status,
hotel_db=#     r.id_room,
hotel_db=#     r.number_room,
hotel_db=#     r.status_room
hotel_db=# FROM hotel.booking_order b
hotel_db=# JOIN hotel.room r
hotel_db=#     ON b.id_room = r.id_room
[hotel_db=# WHERE b.id_order = 4;
 id_order | booking_status | id_room | number_room | status_room
-----+-----+-----+-----+-----
         4 | completed     |        5 |         202 | free
(1 row)

```

Триггер 4 из 7

- нельзя добавить оплату больше суммы бронирования

Триггерная функция

```
hotel_db=# CREATE
OR REPLACE FUNCTION hotel.fn_check_payment_not_greater_order() hotel_db=# RETURNS trigger
hotel_db=# LANGUAGE plpgsql hotel_db=# AS $$ hotel_db=# DECLARE hotel_db=# v_order_amount
numeric;
hotel_db=# BEGIN hotel_db=#
SELECT
amount hotel_db=# INTO v_order_amount hotel_db=#
FROM hotel.booking_order hotel_db=#
WHERE id_order = NEW.id_order;
hotel_db=# hotel_db=# IF v_order_amount IS NULL THEN hotel_db=# RAISE EXCEPTION 'Для
бронирования % не указана сумма.',
NEW.id_order;
hotel_db=# END IF;
hotel_db=# hotel_db=# IF NEW.amount > v_order_amount THEN hotel_db=# RAISE EXCEPTION
'Сумма оплаты % превышает сумму бронирования %.',
hotel_db=# NEW.amount,
v_order_amount;
hotel_db=# END IF;
hotel_db=# hotel_db=# RETURN NEW;
hotel_db=# END;
hotel_db=# $$;
CREATE FUNCTION
```

```
hotel_db=# CREATE OR REPLACE FUNCTION hotel.fn_check_payment_not_greater_order()
hotel_db=# RETURNS trigger
hotel_db=# LANGUAGE plpgsql
hotel_db=# AS $$
hotel_db=# DECLARE
hotel_db=#     v_order_amount numeric;
hotel_db=# BEGIN
hotel_db=#     SELECT amount
hotel_db=#     INTO v_order_amount
hotel_db=#     FROM hotel.booking_order
hotel_db=#     WHERE id_order = NEW.id_order;
hotel_db=#     IF v_order_amount IS NULL THEN
hotel_db=#         RAISE EXCEPTION 'Для бронирования % не указана сумма.', NEW.id_order;
hotel_db=#     END IF;
hotel_db=#     IF NEW.amount > v_order_amount THEN
hotel_db=#         RAISE EXCEPTION 'Сумма оплаты % превышает сумму бронирования %.',
hotel_db=#         NEW.amount, v_order_amount;
hotel_db=#     END IF;
hotel_db=#     RETURN NEW;
hotel_db=# END;
hotel_db=# $$;
CREATE FUNCTION
```

Триггер

```
hotel_db=# CREATE TRIGGER trg_check_payment_not_greater_order
hotel_db=# BEFORE INSERT OR UPDATE ON hotel.payment
hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_check_payment_not_greater_order();
CREATE TRIGGER
hotel_db=#
```

```
CREATE TRIGGER trg_check_payment_not_greater_order
BEFORE INSERT OR UPDATE ON hotel.payment
FOR EACH ROW
EXECUTE PROCEDURE hotel.fn_check_payment_not_greater_order();
```

Состояние после триггера

```

hotel_db=# INSERT INTO hotel.payment (
hotel_db(#      id_payment,
hotel_db(#      id_order,
hotel_db(#      amount,
hotel_db(#      payment_date,
hotel_db(#      payment_method
hotel_db(# )
hotel_db=# VALUES (
hotel_db(#      4,
hotel_db(#      4,
hotel_db(#      20000,
hotel_db(#      CURRENT_DATE,
hotel_db(#      'card'
hotel_db(# );
ERROR: Сумма оплаты 20000 превышает сумму бронирования 16537.50.
CONTEXT: PL/pgSQL function hotel.fn_check_payment_not_greater_order() line 15 at RAISE
hotel_db=#

```

Проверка работы триггера `trg_check_payment_not_greater_order`.

Для проверки была выполнена попытка добавить оплату по бронированию `id_order = 4` на сумму 20000, при этом сумма самого бронирования составляет 16537.50.

В результате сработал триггер `trg_check_payment_not_greater_order`, который вызвал ошибку:

ERROR: Сумма оплаты 20000 превышает сумму бронирования 16537.50. Таким образом, триггер корректно запрещает добавление оплаты, сумма которой превышает сумму бронирования, и обеспечивает целостность данных в таблице `hotel.payment`.

Проверка корректной оплаты, которая меньше суммы бронирования

```

hotel_db=# SELECT
hotel_db(#      id_payment,
hotel_db(#      id_order,
hotel_db(#      amount,
hotel_db(#      payment_date,
hotel_db(#      payment_method
hotel_db=# FROM hotel.payment
hotel_db=# WHERE id_payment = 4;
id_payment | id_order | amount | payment_date | payment_method
-----+-----+-----+-----+-----
4 |      4 | 10000 | 2026-06-11 | card
(1 row)

```

Дополнительно была выполнена проверка корректной оплаты. В таблицу `hotel.payment` была добавлена запись об оплате по бронированию `id_order = 4` на сумму 10000, что меньше суммы бронирования 16537.50.

Запрос выполнен успешно:

```
INSERT 0 1
```

После этого была выполнена проверка таблицы `hotel.payment`, в результате которой появилась запись с `id_payment = 4`, `id_order = 4`, суммой 10000, датой оплаты и способом оплаты `card`.

Таким образом, триггер запрещает только некорректные оплаты, превышающие сумму бронирования, а корректные оплаты допускает.

Триггер 5 из 7

- запретить добавление бронирования на номер, который уже имеет статус booked или occupied.

Триггерная функция

```
hotel_db=# CREATE
OR REPLACE FUNCTION hotel.fn_check_room_available_before_booking() hotel_db - #
RETURNS trigger hotel_db - # LANGUAGE plpgsql hotel_db - # AS $$ hotel_db$ #
DECLARE hotel_db$ # v_room_status character varying;
hotel_db$ # BEGIN hotel_db$ # IF NEW.id_room IS NOT NULL hotel_db$ #
AND NEW.status IN ('created', 'confirmed') THEN hotel_db$ # hotel_db$ #
SELECT
status_room hotel_db$ # INTO v_room_status hotel_db$ #
FROM hotel.room hotel_db$ #
WHERE id_room = NEW.id_room;
hotel_db$ # hotel_db$ # IF v_room_status IN ('booked', 'occupied') THEN hotel_db$ # RAISE
EXCEPTION 'Номер % недоступен для бронирования. Текущий статус: %.',
hotel_db$ # NEW.id_room,
v_room_status;
hotel_db$ # END IF;
hotel_db$ # hotel_db$ # END IF;
hotel_db$ # hotel_db$ # RETURN NEW;
hotel_db$ # END;
hotel_db$ # $$;
CREATE FUNCTION
```

```
hotel_db=# CREATE OR REPLACE FUNCTION hotel.fn_check_room_available_before_booking()
hotel_db=# RETURNS trigger
hotel_db=# LANGUAGE plpgsql
hotel_db=# AS $$
hotel_db$# DECLARE
hotel_db$#     v_room_status character varying;
hotel_db$# BEGIN
hotel_db$#     IF NEW.id_room IS NOT NULL
hotel_db$#         AND NEW.status IN ('created', 'confirmed') THEN
hotel_db$#
hotel_db$#         SELECT status_room
hotel_db$#             INTO v_room_status
hotel_db$#             FROM hotel.room
hotel_db$#             WHERE id_room = NEW.id_room;
hotel_db$#
hotel_db$#         IF v_room_status IN ('booked', 'occupied') THEN
hotel_db$#             RAISE EXCEPTION 'Номер % недоступен для бронирования. Текущий статус: %.',
hotel_db$#                 NEW.id_room, v_room_status;
hotel_db$#         END IF;
hotel_db$#     END IF;
hotel_db$#     RETURN NEW;
hotel_db$# END;
hotel_db$# $$;
CREATE FUNCTION
hotel_db=#
```

Триггер

```
hotel_db=# CREATE TRIGGER trg_check_room_available_before_booking
hotel_db=# BEFORE INSERT OR UPDATE OF id_room, status ON hotel.booking_order
```

```

hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_check_room_available_before_booking();
CREATE TRIGGER

```

Триггер добавлен

Triggers:

```

    trg_check_room_available_before_booking BEFORE INSERT OR UPDATE OF id_room,
status ON hotel.booking_order FOR EACH ROW EXECUTE FUNCTION
hotel.fn_check_room_available_before_booking()
    trg_set_room_booked_after_booking AFTER INSERT ON hotel.booking_order FOR EACH
ROW EXECUTE FUNCTION hotel.fn_set_room_booked_after_booking()
    trg_set_room_free_after_cancel_booking AFTER UPDATE OF status ON
hotel.booking_order FOR EACH ROW EXECUTE FUNCTION
hotel.fn_set_room_free_after_cancel_booking()
    trg_set_room_free_after_complete_booking AFTER UPDATE OF status ON
hotel.booking_order FOR EACH ROW EXECUTE FUNCTION
hotel.fn_set_room_free_after_complete_booking()

```

После триггера

```

hotel_db=# INSERT INTO hotel.booking_order (
hotel_db(#      id_order,
hotel_db(#      payment_type,
hotel_db(#      id_guest,
hotel_db(#      id_employee,
hotel_db(#      amount,
hotel_db(#      check_in_date,
hotel_db(#      check_out_date,
hotel_db(#      status,
hotel_db(#      id_room
hotel_db(# )
hotel_db=# VALUES (
hotel_db(#      6,
hotel_db(#      'card',
hotel_db(#      1,
hotel_db(#      1,
hotel_db(#      5000,
hotel_db(#      DATE '2026-07-01',
hotel_db(#      DATE '2026-07-03',
hotel_db(#      'created',
hotel_db(#      2
[hotel_db(# );
ERROR:  Номер 2 недоступен для бронирования. Текущий статус: booked.
CONTEXT:  PL/pgSQL function hotel.fn_check_room_available_before_booking() line 14 at RAISE
hotel_db=# █

```

```

hotel_db=# INSERT INTO hotel.booking_order (
hotel_db(#      id_order,
hotel_db(#      payment_type,
hotel_db(#      id_guest,
hotel_db(#      id_employee,
hotel_db(#      amount,
hotel_db(#      check_in_date,
hotel_db(#      check_out_date,
hotel_db(#      status,
hotel_db(#      id_room
hotel_db(# )
hotel_db=# VALUES (
hotel_db(#      6,
hotel_db(#      'card',
hotel_db(#      1,
hotel_db(#      1,
hotel_db(#      5000,
hotel_db(#      DATE '2026-07-01',
hotel_db(#      DATE '2026-07-03',

```



```
hotel_db(# 'created',
hotel_db(# 2
hotel_db(# );
ERROR: Номер 2 недоступен для бронирования. Текущий статус: booked.
CONTEXT: PL/pgSQL function hotel.fn_check_room_available_before_booking() line 14 at RAISE
hotel_db=#
```

Проверка работы триггера `trg_check_room_available_before_booking`.

Для проверки была выполнена попытка добавить новое бронирование на номер `id_room = 2`, который уже имеет статус `booked`.

При выполнении команды `INSERT INTO hotel.booking_order` сработал триггер `trg_check_room_available_before_booking`, который вызвал ошибку:

ERROR: Номер 2 недоступен для бронирования. Текущий статус: booked.

Таким образом, триггер корректно запрещает создание бронирования на номер, который уже забронирован или занят, и предотвращает нарушение целостности данных в базе данных гостиницы.

триггер 6 из 7

- запрет пересечения периодов цен для одного типа номера

Для одного и того же типа номера нельзя добавить новую цену, если её период действия пересекается с уже существующим периодом цены.

Сначала создаём триггерную функцию.

Функция

```
CREATE
OR REPLACE FUNCTION hotel.fn_check_price_period_overlap() RETURNS trigger LANGUAGE
plpgsql AS $$ BEGIN IF EXISTS (
SELECT
1
FROM hotel.price p
WHERE p.id_room_type = NEW.id_room_type
AND p.id_price <> NEW.id_price
AND NEW.start_date <= COALESCE(p.end_date, DATE '9999-12-31')
AND COALESCE(NEW.end_date, DATE '9999-12-31') >= p.start_date
) THEN RAISE EXCEPTION 'Для типа номера % уже существует цена на пересекающийся период.',
NEW.id_room_type;
END IF;
RETURN NEW;
END;
$$;
```

```

hotel_db=# CREATE
hotel_db=# OR REPLACE FUNCTION hotel.fn_check_price_period_overlap() RETURNS trigger LANGUAGE plpgsql AS $$ BEGI
N IF EXISTS (
hotel_db$# SELECT
hotel_db$# 1
hotel_db$# FROM hotel.price p
hotel_db$# WHERE p.id_room_type = NEW.id_room_type
hotel_db$# AND p.id_price <> NEW.id_price
hotel_db$# AND NEW.start_date <= COALESCE(p.end_date, DATE '9999-12-31')
hotel_db$# AND COALESCE(NEW.end_date, DATE '9999-12-31') >= p.start_date
hotel_db$# ) THEN RAISE EXCEPTION 'Для типа номера % уже существует цена на пересекающийся период.',
hotel_db$# NEW.id_room_type;
hotel_db$# END IF;
hotel_db$# RETURN NEW;
hotel_db$# END;
hotel_db$# $$;

```

Создание триггера

```

hotel_db=# CREATE TRIGGER trg_check_price_period_overlap
hotel_db=# BEFORE INSERT OR UPDATE ON hotel.price
hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_check_price_period_overlap();
CREATE TRIGGER

```

```

CREATE TRIGGER trg_check_price_period_overlap
BEFORE INSERT OR UPDATE ON hotel.price
FOR EACH ROW
EXECUTE PROCEDURE hotel.fn_check_price_period_overlap();

```

Проверка, что он появился у таблицы price

Triggers:

```

trg_check_price_period_overlap BEFORE INSERT OR UPDATE ON hotel.price FOR EACH
ROW EXECUTE FUNCTION hotel.fn_check_price_period_overlap()

```

Проверка работы триггера trg_check_price_period_overlap.

Для проверки была выполнена попытка добавить новую цену для типа номера id_room_type = 1 на период с 2026-01-01 по 2026-12-31.

Так как для данного типа номера уже существует цена на пересекающийся период, сработал триггер trg_check_price_period_overlap, который вызвал ошибку:

ERROR: Для типа номера 1 уже существует цена на пересекающийся период. Таким образом, триггер корректно запрещает добавление пересекающихся периодов действия цен для одного и того же типа номера и обеспечивает целостность данных в таблице hotel.price.

```

hotel_db=# INSERT INTO hotel.price(id_price, start_date, end_date, price_per_day, id_room_type)
hotel_db=# VALUES
hotel_db=# (4, DATE '2026-01-01', DATE '2026-12-31', 4000, 1);
ERROR: Для типа номера 1 уже существует цена на пересекающийся период.
CONTEXT: PL/pgSQL function hotel.fn_check_price_period_overlap() line 9 at RAISE
hotel_db=#

```

```

INSERT INTO hotel.price(id_price, start_date, end_date, price_per_day, id_room_type)
VALUES
(4, DATE '2026-01-01', DATE '2026-12-31', 4000, 1);

```

Триггер 7 из 7.

- проверка корректности акции

Смысл: при добавлении или изменении акции нельзя допустить:

- скидку меньше 1% или больше 100%;
- дату окончания раньше даты начала.

Функция

```
hotel_db=# CREATE
hotel_db=# OR REPLACE FUNCTION hotel.fn_check_promotion_correct() RETURNS trigger LANGUAGE plpgsql AS $$ BEGIN IF NEW.discount_percent <= 0
hotel_db=# OR NEW.discount_percent > 100 THEN RAISE EXCEPTION 'Процент скидки должен быть больше 0 и не больше 100. Указано: %.',
hotel_db=# NEW.discount_percent;
hotel_db=# END IF;
hotel_db=# IF NEW.start_date >= NEW.end_date THEN RAISE EXCEPTION 'Дата начала акции должна быть раньше даты окончания.';
hotel_db=# END IF;
hotel_db=# RETURN NEW;
hotel_db=# END;
hotel_db=# $$;
CREATE FUNCTION
hotel_db=#
```

CREATE

```
OR REPLACE FUNCTION hotel.fn_check_promotion_correct() RETURNS trigger LANGUAGE plpgsql
AS $$ BEGIN IF NEW.discount_percent <= 0
OR NEW.discount_percent > 100 THEN RAISE EXCEPTION 'Процент скидки должен быть больше 0
и не больше 100. Указано: %.',
NEW.discount_percent;
END IF;
IF NEW.start_date >= NEW.end_date THEN RAISE EXCEPTION 'Дата начала акции должна быть
раньше даты окончания.';
END IF;
RETURN NEW;
END;
$$;
```

Триггер

```
CREATE TRIGGER trg_check_promotion_correct
BEFORE INSERT OR UPDATE ON hotel.promotion
FOR EACH ROW
EXECUTE PROCEDURE hotel.fn_check_promotion_correct();
```

```
hotel_db=# CREATE TRIGGER trg_check_promotion_correct
hotel_db=# BEFORE INSERT OR UPDATE ON hotel.promotion
hotel_db=# FOR EACH ROW
hotel_db=# EXECUTE PROCEDURE hotel.fn_check_promotion_correct();
CREATE TRIGGER
hotel_db=#
```

Проверка и результаты

Проверка работы триггера trg_check_promotion_correct.

Для проверки была выполнена попытка добавить новую акцию в таблицу hotel.promotion со скидкой 150%.

Такое значение является некорректным, так как процент скидки должен быть больше 0 и не больше 100. При выполнении команды INSERT INTO hotel.promotion сработал триггер trg_check_promotion_correct, который вызвал ошибку:

ERROR: Процент скидки должен быть больше 0 и не больше 100. Указано: 150.

Таким образом, триггер корректно запрещает добавление акций с некорректным процентом скидки и обеспечивает целостность данных в таблице hotel.promotion.

```
hotel_db=# INSERT INTO hotel.promotion(id_promotion, name, discount_percent, start_date, end_date, id_room_type, conditions)
hotel_db=# VALUES
hotel_db=# (4, 'Ошибка скидки', 150, DATE '2026-07-01', DATE '2026-07-31', 1, 'Тест некорректной скидки');
ERROR: Процент скидки должен быть больше 0 и не больше 100. Указано: 150.
CONTEXT: PL/pgSQL function hotel.fn_check_promotion_correct() line 2 at RAISE
hotel_db=#
```

Вывод

В ходе выполнения лабораторной работы были изучены и применены на практике процедуры, функции и триггеры в PostgreSQL.

Для индивидуальной базы данных гостиницы были созданы три хранимые процедуры: для увеличения стоимости номеров при отсутствии свободных номеров, для получения информации о свободных одноместных номерах на завтрашний день и для бронирования двухместного номера на заданную дату и количество дней проживания.

Также были созданы семь триггеров, обеспечивающих автоматическое изменение статусов номеров, проверку корректности оплаты, запрет бронирования занятых номеров, контроль пересечения периодов цен и проверку корректности акций.

В процессе работы были использованы триггерные функции с типом возвращаемого значения trigger, специальные переменные NEW и OLD, а также триггеры типов BEFORE и AFTER с уровнем выполнения FOR EACH ROW.

Все созданные процедуры и триггеры были проверены с помощью SQL-запросов в консоли psql. Результаты проверок подтвердили корректность их работы и обеспечение целостности данных в базе данных гостиницы.